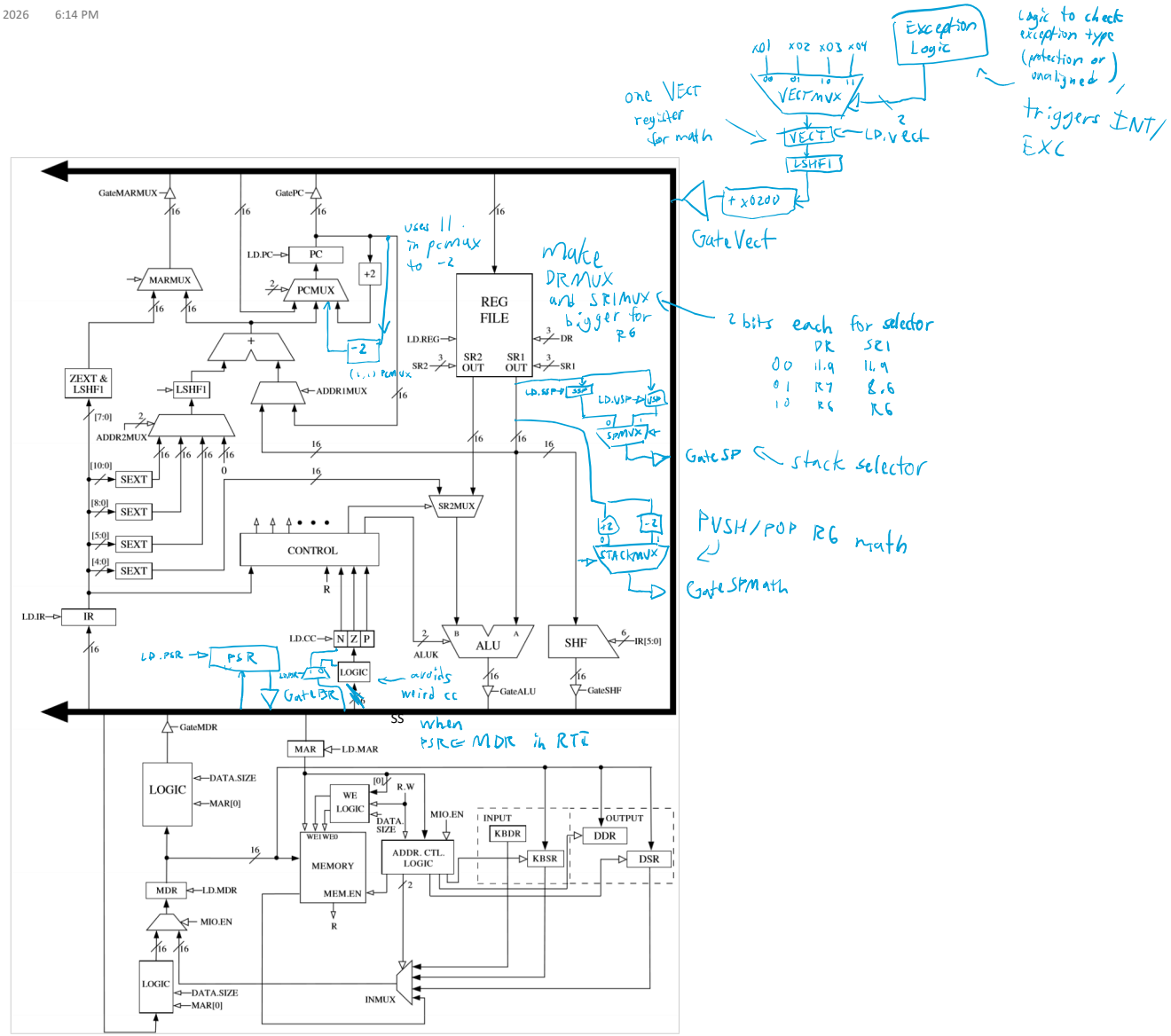
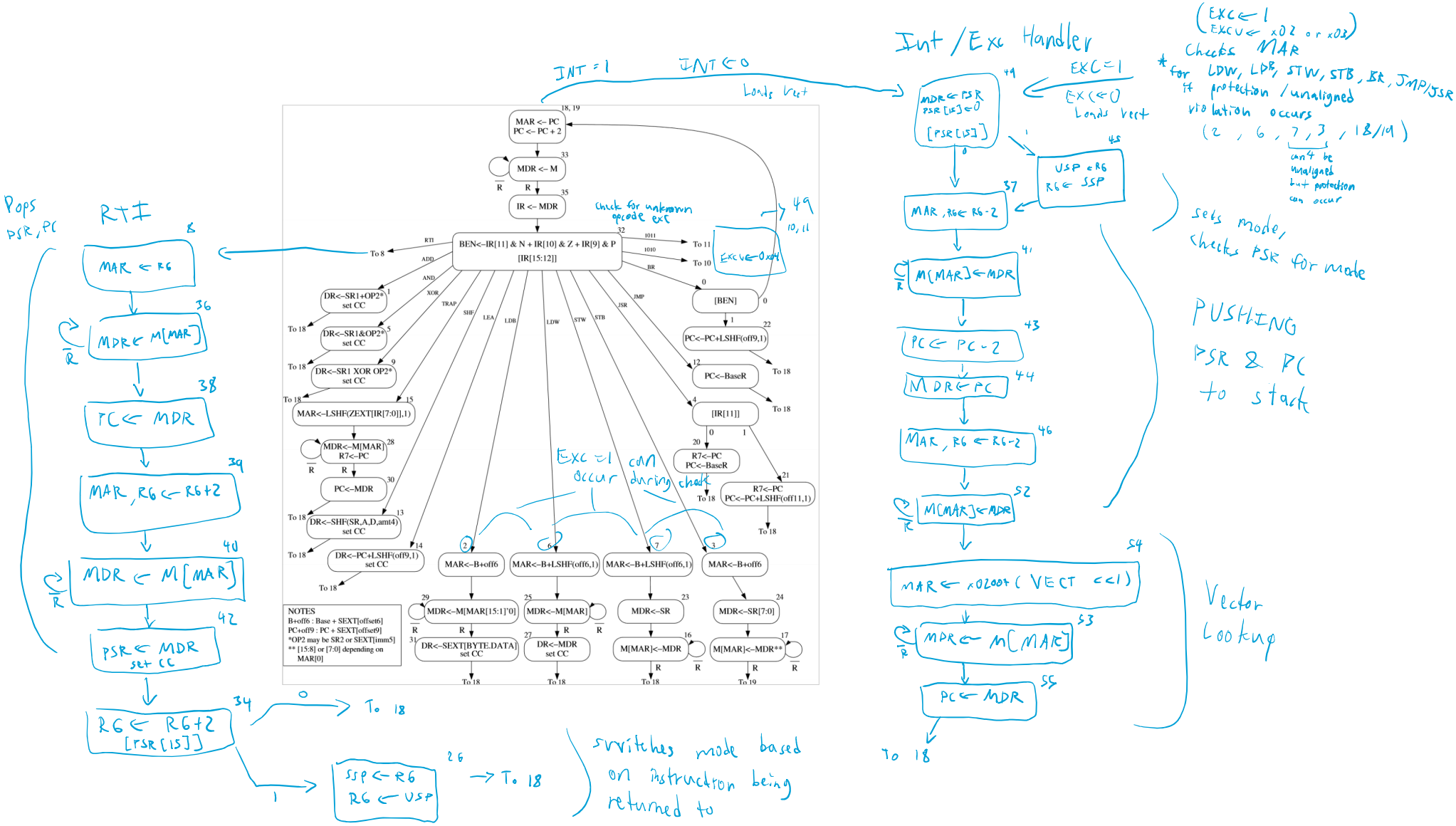
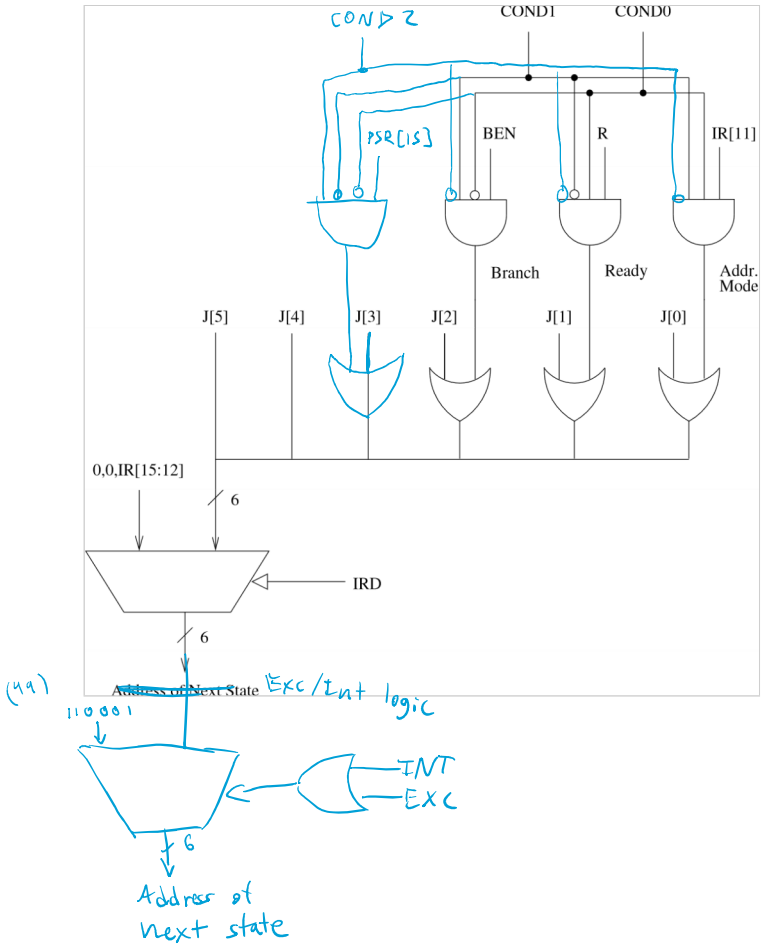
Lab 4

Saturday, March 21, 2026 6:14 PM




INT = 1
INT ← 0
Loads vect
Int / Exc Handler
(EXC ← 1
EXCV ← x02 or x03)
Checks MAR
* for LDW, LDB, STW, STB, BR, JMP/JSR
if protection / unaligned
violation occurs
(2, 6, 7, 3, 18/19)
can't be unaligned but protection can occur
EXC = 1
EXC ← 0
Loads vect
MDR ← PSR
PSR[15] ← 0
[PSR[15]]
USP ← R6
R6 ← SSP
MAR, R6 ← R6 - 2
M[MAR] ← MDR
PC ← PC - 2
MDR ← PC
MAR, R6 ← R6 - 2
M[MAR] ← MDR
MAR ← x0200 + (VECT << 1)
MDR ← M[MAR]
PC ← MDR
To 18
sets mode, checks PSR for mode
PUSHING PSR & PC to stack
Vector Lookup
MAR <- PC
PC <- PC + 2
MDR <- M
IR <- MDR
check for unknown opcode exc
BEN<-IR[11] & N + IR[10] & Z + IR[9] & P
[IR[15:12]]
EXCV ← 0x04
[BEN]
PC<-PC+LSHF(off9,1)
PC<-BaseR
[IR[11]]
R7<-PC
PC<-BaseR
R7<-PC
PC<-PC+LSHF(off11,1)
DR<-SR1+OP2*
set CC
DR<-SR1&OP2*
set CC
DR<-SR1 XOR OP2*
set CC
MAR<-LSHF(ZEXT[IR[7:0]],1)
MDR<-M[MAR]
R7<-PC
PC<-MDR
DR<-SHF(SR,A,D,amt4)
set CC
DR<-PC+LSHF(off9,1)
set CC
MAR<-B+off6
MAR<-B+LSHF(off6,1)
MAR<-B+LSHF(off6,1)
MAR<-B+off6
MDR<-M[MAR[15:1]'0]
DR<-SEXT[BYTE.DATA]
set CC
MDR<-M[MAR]
DR<-MDR
set CC
MDR<-SR
M[MAR]<-MDR
MDR<-SR[7:0]
M[MAR]<-MDR**
NOTES
B+off6 : Base + SEXT[offset6]
PC+off9 : PC + SEXT[offset9]
*OP2 may be SR2 or SEXT[imm5]
** [15:8] or [7:0] depending on MAR[0]
Exc = 1 occur
can during check
RTI
Pops PSR, PC
MAR ← R6
MDR ← M[MAR]
PC ← MDR
MAR, R6 ← R6+2
MDR ← M[MAR]
PSR ← MDR
set CC
R6 ← R6+2
[PSR[15]]
SSP ← R6
R6 ← USP
switches mode based on instruction being returned to



I added a new COND bit that checks PSR 15 bit to check which mode the user was in during the int/exc. I also added a mux to the end that will set the J bits to state 49 when the INT or EXC signal is triggered.

IRD	Cond	J	LD.MAR	LD.MDR	LD.IR	LD.BEN	LD.REG	LD.CC	LD.PC	GatePC	GateMDR	GateALU	GateMARMUX	GateSHF	PCMUX	DRMUX0	SR1MUX0	ADDR1MUX	ADDR2MUX	MARMUX	ALUK	MIO.EN	R.W	DATA.SIZE	LSHF1	GatePSR	GateVect	GateSP	GateSPMath	LD.PSR	LD.SSP	LD.USP	LD.Vect	SPMUX	STACKMUX	Cond2	DRMUX1	SR1MUX1

new

I added new gate signals for the PSR, Vector Math, SP selector, and SP push/pop math for when they need to send data to the bus in the new states. I added LD.PSR for RTI when MDR loads PSR register. Because this would mess up the condition codes, I have a selector MUX in the datapath that bypasses that logic and just updates with the exact old PSR from MDR and the ld.psr is the selector signal for that. LD.SSP and LD.USP are for when R6 loads those. LD.Vect is used on the states where INT and EXC are checked so the VECT register can be loaded with whatever type of interrupt it is. SPMUX selects between putting the SSP or the USP register on the bus. STACKMUX selects between adding and subtracting R6 for the stack. I added an extra COND bit for PSR[15] checking and I increased the DRMUX and SR1MUX to two bits to select R6 at 10. Also, PCMUX uses 11 now to subtract 2.